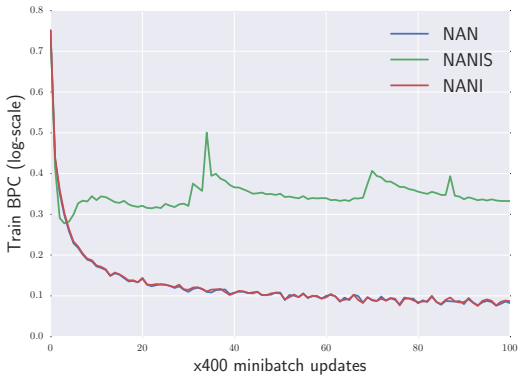

Figure 8. We show the learning curves of a simple character-level GRU language model over the sequences of length 200 on PTB. NANI and NAN, have very similar learning curves. NANIS in the beginning of the training has a better progress than NAN and NANIS, but then training curve stops improving.

Table 1. Performance of the noisy network on the *Learning to Execute* / task. Just changing the activation function to the proposed noisy one yielded about 2.5% improvement in accuracy.

Model name	Test Accuracy
Reference Model	46.45%
Noisy Network(**NAH**)	**48.09%**

the default gradient clipping to 5 from 10 in order to avoid numerical stability problems. When evaluating a network, the length (number of lines) of the executed programs was set to 6 and nesting was set to 3, which are default settings in the released code for these tasks. Both the reference model and the model with noisy activations were trained with "combined" curriculum which is the most sophisticated and the best performing one.

Our results show that applying the proposed activation function leads to better performance than that of the reference model. Moreover it shows that the method is easy to combine with a non-trivial learning curriculum. The results are presented in Table 1 and in Figure 10

6.3. Penntreebank Experiments

We trained a 2−layer word-level LSTM language model on Penntreebank. We used the same model proposed by Zaremba et al. (2014). [2]We simply replaced all sigmoid and tanh units with noisy hard-sigmoid and hard-tanh units. The reference model is a well-finetuned strong baseline from (Zaremba et al., 2014). For the noisy experiments we used exactly the same setting, but decreased the gradient clipping threshold to 5 from 10. We provide the results of different

[2]We used the code provided in https://github.com/wojzaremba/lstm

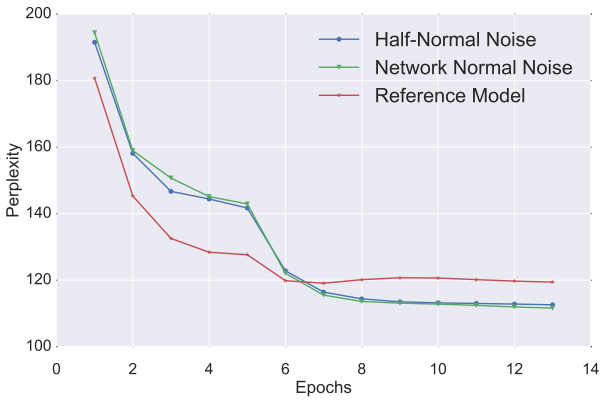

Figure 9. Learning curves of validation perplexity for the LSTM language model on word level on Penntreebank dataset.

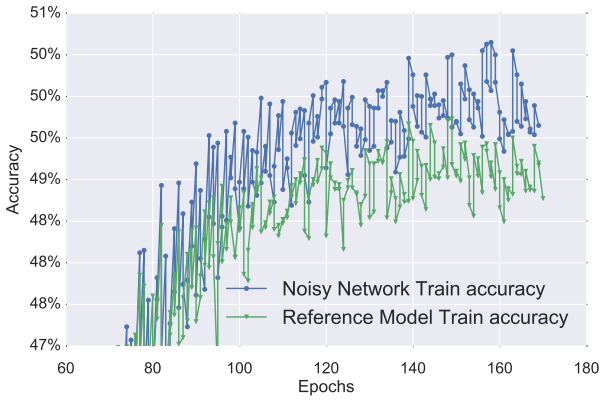

Figure 10. Training curves of the reference model (Zaremba & Sutskever, 2014) and its noisy variant on the "Learning To Execute" problem. The noisy network converges faster and reaches a higher accuracy, showing that the noisy activations help to better optimize for such hard to optimize tasks.

models in Table 2. In terms of validation and test performance we did not observe big difference between the additive noise from Normal and half-Normal distributions, but there is a substantial improvement due to noise, which makes this result the new state-of-the-art on this task, as far as we know.

6.4. Neural Machine Translation Experiments

We have trained a neural machine translation (NMT) model on the Europarl dataset with the neural attention model (Bahdanau et al., 2014).[3] We have replaced all sigmoid and tanh units with their noisy counterparts. We have scaled down the weight matrices initialized to be orthogonal scaled by multiplying with 0.01. Evaluation is done on the newstest2011 test set. All models are trained

[3]Again, we have used existing code, provided in https://github.com/kyunghyuncho/dl4mt-material, and only changed the nonlinearities

Table 3. Image Caption Generation on Flickr8k. This time we added noisy activations in the code from (Xu et al., 2015) and obtain substantial improvements on the higher-order BLEU scores and the METEOR metric, as well as in NLL. Soft attention and hard attention here refers to using backprop versus REINFORCE when training the attention mechanism. We fixed $\sigma = 0.05$ for NANI and $c = 0.5$ for both NAN and NANIL.

	BLEU -1	BLEU-2	BLEU-3	BLEU-4	METEOR	Test NLL
Soft Attention (Sigmoid and Tanh) (Reference)	**67**	44.8	29.9	19.5	18.9	40.33
Soft Attention (**NAH Sigmoid & Tanh**)	66	**45.8**	**30.69**	**20.9**	**20.5**	40.17
Soft Attention (**NAH Sigmoid & Tanh** wo dropout)	64.9	44.2	**30.7**	**20.9**	20.3	**39.8**
Soft Attention (**NANI Sigmoid & Tanh**)	66	45.0	30.6	20.7	**20.5**	40.0
Soft Attention (**NANIL Sigmoid & Tanh**)	66	44.6	30.1	20.0	**20.5**	39.9
Hard Attention (Sigmoid and Tanh)	67	45.7	31.4	21.3	19.5	-

Table 2. Penntreebank word-level comparative perplexities. We only replaced in the code from Zaremba et al. (2014) the sigmoid and tanh by corresponding noisy variants and observe a substantial improvement in perplexity, which makes this the state-of-the-art on this task.

	Valid ppl	Test ppl
Noisy LSTM + **NAN**	111.7	**108.0**
Noisy LSTM + **NAH**	112.6	**108.7**
LSTM (Reference)	119.4	115.6

Table 4. Neural machine Translation on Europarl. Using existing code from (Bahdanau et al., 2014) with nonlinearities replaced by their noisy versions, we find much improved performance (2 BLEU points is considered a lot for machine translation). We also see that simply using the hard versions of the nonlinearities buys about half of the gain.

	Valid nll	BLEU
Sigmoid and Tanh NMT (Reference)	65.26	20.18
Hard-Tanh and Hard-Sigmoid NMT	64.27	21.59
Noisy (**NAH**) Tanh and Sigmoid NMT	**63.46**	**22.57**

with early-stopping. We also compare with a model with hard-tanh and hard-sigmoid units and our model using noisy activations was able to outperform both, as shown in Table 4. Again, we see a substantial improvement (more than 2 BLEU points) with respect to the reference for English to French machine translation.

6.5. Image Caption Generation Experiments

We evaluated our noisy activation functions on a network trained on the Flickr8k dataset. We used the soft neural attention model proposed in (Xu et al., 2015) as our reference model.[4] We scaled down the weight matrices initialized to be orthogonal scaled by multiplying with 0.01. As shown in Table 3, we were able to obtain better results than the reference model and our model also outperformed the best model provided in (Xu et al., 2015) in terms of Meteor score.

(Xu et al., 2015)'s model was using dropout with the ratio of 0.5 on the output of the LSTM layers and the context. We have tried both with and without dropout, as in Table 3, we observed improvements with the addition of dropout to the noisy activation function. But the main improvement seems to be coming with the introduction of the noisy activation functions since the model without dropout already outperforms the reference model.

6.6. Experiments with Continuation

We performed experiments to validate the effect of annealing the noise to obtain a continuation method for neural networks.

We designed a new task where, given a random sequence of integers, the objective is to predict the number of unique elements in the sequence. We use an LSTM network over the input sequence, and performed a time average pooling over the hidden states of LSTM to obtain a fixed-size vector. We feed the pooled LSTM representation into a simple (one hidden-layer) ReLU MLP in order to predict the unique number of elements in the input sequence. In the experiments we fixed the length of input sequence to 26 and the input values are between 0 and 10. In order to anneal the noise, we started training with the scale hyperparameter of the standard deviation of noise with $c = 30$ and annealed it down to 0.5 with the schedule of $\frac{c}{\sqrt{t+1}}$ where t is being incremented at every 200 minibatch updates. When noise annealing is combined with a curriculum strategy (starting with short sequences first and gradually increase the length of the training sequences), the best models are obtained.

As a second test, we used the same annealing procedure in order to train a Neural Turing Machine (NTM) on the associative recall task (Graves et al., 2014). We trained our model with a minimum of 2 items and a maximum of 16 items. We show results of the NTM with noisy activations in the controller, with annealed noise, and compare with a regular NTM in terms of validation error. As can be seen in Figure 11, the network using noisy activation converges much faster and nails the task, whereas the original network failed to approach a low error.

[4] We used the code provided at https://github.com/kelvinxu/arctic-captions.

Table 5. Experimental results on the task of finding the unique number of elements in a random integer sequence. This illustrates the effect of annealing the noise level, turning the training procedure into a continuation method. Noise annealing yields better results than the curriculum.

	Test Error %
LSTM+MLP(Reference)	33.28
Noisy LSTM+MLP(**NAN**)	31.12
Curriculum LSTM+MLP	14.83
Noisy LSTM+MLP(**NAN**) Annealed Noise	**9.53**
Noisy LSTM+MLP(**NANIL**) Annealed Noise	20.94

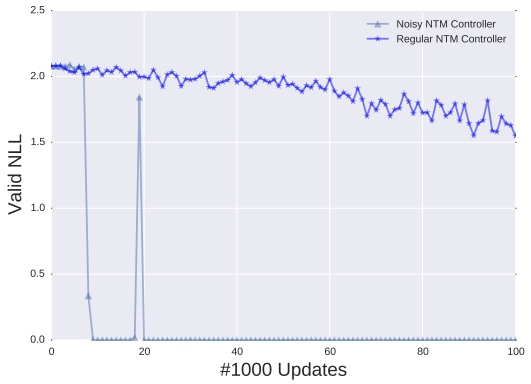

Figure 11. Validation learning curve of NTM on Associative recall task evaluated over items of length 2 and 16. The NTM with noisy controller converges much faster and solves the task.

7. Conclusion

Nonlinearities in neural networks are both a blessing and a curse. A blessing because they allow to represent more complicated functions and a curse because that makes the optimization more difficult. For example, we have found in our experiments that using a hard version (hence more nonlinear) of the sigmoid and $\tanh$ nonlinearities often improved results. In the past, various strategies have been proposed to help deal with the difficult optimization problem involved in training some deep networks, including curriculum learning, which is an approximate form of continuation method. Earlier work also included softened versions of the nonlinearities that are gradually made harder during training. Motivated by this prior work, we introduce and formalize the concept of noisy activations as a general framework for injecting noise in nonlinear functions so that large noise allows SGD to be more exploratory. We propose to inject the noise to the activation functions either at the input of the function or at the output where unit would otherwise saturate, and allow gradients to flow even in that case. We show that our noisy activation functions are easier to optimize. It also, achieves better test errors, since the noise injected to the activations also regularizes the model as well. Even with a fixed noise level, we found the proposed noisy activations to outperform their sigmoid or $\tanh$ counterpart on different tasks and datasets, yielding state-of-the-art or competitive results with a simple modification, for example on PennTreebank. In addition, we found that annealing the noise to obtain a continuation method could further improved performance.

References

Allgower, E. L. and Georg, K. *Numerical Continuation Methods. An Introduction*. Springer-Verlag, 1980.

Bahdanau, Dzmitry, Cho, Kyunghyun, and Bengio, Yoshua. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

Bengio, Yoshua. Estimating or propagating gradients through stochastic neurons. Technical Report arXiv:1305.2982, Universite de Montreal, 2013.

Bengio, Yoshua, Louradour, Jerome, Collobert, Ronan, and Weston, Jason. Curriculum learning. In *ICML'09*, 2009.

Bengio, Yoshua, Léonard, Nicholas, and Courville, Aaron. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

Cho, Kyunghyun, Van Merriënboer, Bart, Gulcehre, Caglar, Bahdanau, Dzmitry, Bougares, Fethi, Schwenk, Holger, and Bengio, Yoshua. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

Ge, Rong, Huang, Furong, Jin, Chi, and Yuan, Yang. Escaping from saddle points—online stochastic gradient for tensor decomposition. *arXiv preprint arXiv:1503.02101*, 2015.

Glorot, Xavier, Bordes, Antoine, and Bengio, Yoshua. Deep sparse rectifier neural networks. In *International Conference on Artificial Intelligence and Statistics*, pp. 315–323, 2011.

Goodfellow, Ian J, Warde-Farley, David, Mirza, Mehdi, Courville, Aaron, and Bengio, Yoshua. Maxout networks. *arXiv preprint arXiv:1302.4389*, 2013.

Graves, Alex, Wayne, Greg, and Danihelka, Ivo. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

Hannun, Awni, Case, Carl, Casper, Jared, Catanzaro, Bryan, Diamos, Greg, Elsen, Erich, Prenger, Ryan, Satheesh, Sanjeev, Sengupta, Shubho, Coates, Adam, et al. Deep speech: Scaling up end-to-end speech recognition. *arXiv preprint arXiv:1412.5567*, 2014.

Hermann, Karl Moritz, Kocisky, Tomas, Grefenstette, Edward, Espeholt, Lasse, Kay, Will, Suleyman, Mustafa, and Blunsom, Phil. Teaching machines to read and comprehend. In *Advances in Neural Information Processing Systems*, pp. 1684–1692, 2015.